CCIT Report #540 July 2005

Efficient Link Capacity and QoS Design for Wormhole Network-on-Chip

Zvika Guz[1], Isask'har Walter[1], Evgeny Bolotin[1], Israel Cidon[2], Ran Ginosar[2] and Avinoam Kolodny[2]
Electrical Engineering Department, Technion – Israel Institute of Technology, Haifa 32000, Israel
[1]{zguz, zigi, bolotin}@tx.technion.ac.il [2]{cidon, ran, kolodny}@ee.technion.ac.il

Abstract

The allocation of link capacities is an important phase in the automated design process of a network on chip based system. Communication resources must be efficiently allocated so that Quality of Service requirements are met but total network cost is minimized.

The paper contributions are twofold: First we introduce a novel analytical delay model for virtual channelled wormhole networks with non-uniform link capacities that eliminates costly simulations in the inner-loop of the optimization process. Second, we present an efficient capacity allocation algorithm that assigns link capacities so that packets of each flow arrive within the specified time. Using examples, we demonstrate the benefit of capacity allocation for a typical SoC, where the traffic is heterogeneous and delay requirements may largely vary.

1. Introduction

As the density of VLSI designs increases, the number of modules in a typical System on Chip (SoC) is expected to be significantly larger than today. Traditional interconnection schemes might no longer be adequate for these systems. The concept of *Network on a Chip* (NoC), introduced in [1], suggests that different modules would be connected by a simple network of shared links and routers, which would enable them to exchange information using data packets. As was analyzed in [2], NoC power and area costs scale much better than shared bus, segmented bus or dedicated point to point connections. Moreover, [3] has made the observation that NoC introduces a better SoC design methodology as standard network interfaces eliminate the need to redesign modules every time communication needs are globally increased (as opposed to shared busses or dedicated links). Networks that employ wormhole flow-control were introduced in [4] and are in wide use in multi-computer interconnect. In wormhole networks each packet is divided into a sequence of *flit*s which are transmitted over physical links one by one in pipeline fashion. A hop-to-hop credit mechanism assures that a flit is transmitted only when the receiving port has free space in its input buffer. Wormhole routing based networks fit the chip interconnect challenge as they minimize communication latencies, require small buffer space and are simple to implement.

In [5], Bolotin et al. presented the QNoC architecture: a wormhole based network which guarantees quality of service (QoS) of different inter-module traffic types. Another important feature is the ability to construct an asymmetric network with variable speed inter-router links.

NoC is customized through an automatic network design phase for a particular SoC implementation ([3], [5], [6]). The goal of the QNoC design process is to minimize the network cost (in terms of area and power) while maintaining the required QoS for the specific system communication demands ([5]). This customization is based on adequate bandwidth allocation to each of the network links. Insufficient allocation will not meet performance requirements while lavish allocation will result in excessive power and area consumptions.

Previous NoC literature dealt with the network performance mainly by simulations as no adequate analytical model that allows quick assessment of network performance exists. Existing models of wormhole networks ([7]-[14]) have not dealt with both heterogeneous traffic patterns and virtual channels, which are fundamental characteristics of NoC interconnect. The use of simulations makes the task of searching for a low cost and high performance capacity allocation computationally extensive and it does not scale well with the size of the problem. On the other hand, a detailed exact analytical solution of a complex NoC is intractable. Using exclusively simplistic approximations may lead to a fast but inaccurate design.

In this work, we propose a hybrid methodology to the network design problem that combines the best of both worlds. First, we propose a novel and simple analytical delay model for a wormhole based NoC, which approximates the network behavior in a wide range of loads. Given any system (in terms of topology, routing and link capacities) and its communication demands (in terms of packets length and generation rate), the model estimates the delay experienced by every source destination pair. To the best of our knowledge, this is the first modeling and analysis of a wormhole network with non-uniform link capacities. Next, we propose an algorithm that applies the delay modeling and analysis to efficiently allocate capacities to network links without need for repetitive simulations in the inner-loop of optimization ([5]). Simulation runs are only used for final verification and fine tuning of the system. Using a design example we demonstrate that our algorithm considerably decreases the total NoC cost and significantly improves the quality and speed of the customization process.

The rest of this paper is organized as follows: Section 2 reviews related work, Section 3 briefly discusses the QNoC architecture, Section 4 describes our wormhole delay model and derives the appropriate equations, Section 5 presents our capacity allocation algorithm, Section 6 provides a design example and simulation results. Finally, section 7 concludes the paper.

2. Related work

The performance of wormhole networks has been extensively studied in the past. While several analytical studies have evaluated the mean latency of packets in wormhole networks (e.g. [7]-[14]), no previous work analyzes wormhole networks with heterogeneous link capacities, which is a fundamental characteristic of QNoC, making it cost effective ([5]). In addition, no previous work analyzes networks with both virtual-channels and non-uniform source-destination communication patterns. For example, [7] presents an analytical model of a wormhole network with an arbitrary number of virtual channels per link. Though comprehensive, it assumes that messages destinations are uniformly distributed. In [8], the model does reflect non-uniform traffic but does not address networks with multiple virtual channels.

The need to allocate bandwidth to NoC links was introduced in [5] along with a simple allocation heuristic: Each link's bandwidth is separately adjusted such that all network links have equal utilization factor. In order to minimize cost, the highest utilization factor for which all requirements are met is chosen. However, this capacity allocation scheme generally suits systems with flows that have similar delay requirements, and involves multiple simulations. This work suggests an improved and scalable capacity allocation algorithm for a general SoC with arbitrary communication demands and delay requirements,

Similarly to this paper, [15] shows that adjusting NoC resources may lead to significant savings. However, [15] dealt with buffer sizing in store-and-forward networks and cannot be applied to wormhole-based ones. Furthermore, wormhole routing requires only a small amount of buffers in each input port ([2]), hence the potential for reducing cost using any buffer sizing technique is limited.

3. QNoC architecture

The QNoC architecture ([5]) is based on a two dimensional grid topology of irregular mesh and wormhole packet routing (as illustrated figure 1). The network is lossless, and packets traverse the network on a shortest path using XY routing. During the design process, each link's bandwidth is separately adjusted so that packet delay requirements are met. Therefore, different network links, connecting routers to other routers or modules, may have different bandwidth.

High performance wormhole based interconnect systems often include virtual channels (VCs) which increase NoC throughput ([16]). The buffer space on both ends of each link is divided into several logical buffers where each pair (i.e. one in the output port and the matching one in the input port) identifies a separate *channel*. When the head flit of a packet arrives at an output port, it is assigned one of the link channels. The packet will use only this channel, and no other packet can share the channel buffers. If no virtual channel is available, the packet is blocked. Virtual channels must be included in QNoC architecture when links have different capacities to allow the multiplexing of several slow streams over a high bandwidth link (preventing overrun conditions).

The flits of different VCs that contend for the link bandwidth are time-multiplexed according to some arbitration policy. QNoC employs a simple policy in which the flits of the active outgoing VCs are transmitted in a round-robin manner over the physical link.

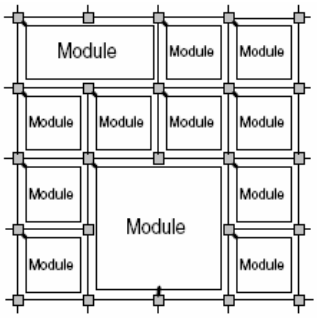


Figure 1: A simple irregular mesh grid topology of a NoC

4. Wormhole delay model

In a wormhole network, the transit time of a packet between a specific source-destination pair is composed of two components ([14]): the time it takes the head flit to reach the destination module (path acquisition time) and the time it takes the rest of the packet to exit the network (transfer time). Predicting the average path acquisition time is a challenging problem due to complex interaction among different flows in the system. Predicting the transfer time is also quite complicated as it may be affected by other flows sharing the same links (link's capacity is time multiplexed among all virtual channels sharing the link).

The suggested model targets on-chip wormhole networks, which are not expected to work under very high utilization rates. Similar to packet switching networks, when a wormhole-based network is highly loaded, the delays become high and long queues accumulate. Moreover, the transfer time of packets of the same flow varies largely as the system becomes very sensitive to temporal conditions. The delay/cost tradeoffs are very different in NoCs in comparison to wide area networks. Generally, SoC cannot tolerate large variances in signal and data transfer times but its link costs are much lower. Therefore, we assume that NoCs will be designed to operate under a relatively light load. Consequently, for the sake of simplicity and computational efficiency, our analysis addresses low to medium loads and does not attempt to achieve high accuracy under very high utilizations.

Previous work ([16]) has showed that adding VCs increases the maximal network throughput. As the silicon cost of VC is moderate, we assume that physical links are split into an adequate number of VCs. In particular, we assume that a head flit acquires a channel instantaneously on every link it traverses.

Our analysis focuses on the transfer of long packets, i.e. packets which are composed of a number of flits significantly larger than the number of buffers along their path. From the simulations presented in [5], it is clear that such packets (termed the *block transfer* class of service) are the ones that place the most stringent demand on NoC resources. Note that the analysis can be easily extended to include packets of any size and to handle multiple priority levels.

We consider a wormhole deadlock free fixed routing network that is composed of N routers connected by unidirectional links. The packets composing each source-destination pair traffic identify a *flow*.

Our model uses the following assumptions:

1. Packets of the same flow have fixed length and are generated by a Poisson process.
2. Sources have infinite queues and sinks immediately consume flits arriving at their destination.
3. Routers have a single flit input queue per VC
4. The propagation delay of flits through links and routers is negligible.
5. Credit signal (back-pressure mechanism) is instantaneous.

4.1. Notation

The network is characterized using the following notations:

C_j = *capacity of link j [bits per second]*

l = *flit size [bits]*

F = *the set of all flows, from every source module* $1 \le s \le N$ *to every destination module* $1 \le d \le N$

f^i = *A flow from the set* F

m^i = *the mean packet length of flow* $f^i \in F$ *[flits]*

λ^i = *average packet generation rate of flow* $f^i \in F$ *[packet/sec]*

π^i = *the set of links composing the fixed path of flow* f^i

π^i_j = *the set of links that are subsequent to link j on flow i's path (a suffix of the path* π^i *)*

The following notations are used in order to analyze the packets delay:

T^i = *the mean transit time of packets of flow* f^i *(the average time elapsed since a packet is created until its last flit exits the network)*

t^i_j = *the mean time to deliver a flit of flow i over link j (waiting for transmission and transmission times)*

Λ^i_j = *the total flit injection rate of all flows sharing link j except flow* f^i *[flits/sec]*.

4.2. Wormhole analysis

The delivery of a packet in a wormhole network resembles a pipeline traversal. When the number of parts composing the packet is considerably larger than the number of pipeline stages, the latency (the time it takes the first bits to exit the pipe) is insignificant compared to the total time, which in this case is mostly affected by the pipeline's throughput. Since packets are assumed to be considerably longer than the buffers along their path, and since each head flit instantaneously acquires a virtual channel on every link it arrives at, we ignore the path acquisition time and approximate the transmission time only.

As in a classic pipeline, the transfer time is dominated by the stage with the smallest service rate. Since flits of different flows are interleaved on links, the approximation of t^i_j should account for the transmission time of other flits on the same link. We use a modification of the basic M/M/1 modeling ([17]) as an approximation of the flit interleaving delay:

$$t^i_j = \frac{1}{\frac{1}{l} \cdot C_j - \Lambda^i_j} \quad (1)$$

where Λ^i_j (defined above) is the bandwidth consumed by all flows other than flow i on link j. Formally:

$$\Lambda^i_j = \sum_{f \mid j \in \pi^f \wedge f \neq i} \lambda^f \cdot m^f \quad (2)$$

Equation (1) models the mean interleaving delay experienced by flits of flow i on link j as a simple queue, without accounting for the bandwidth consumed by packets of flow i itself. This modification is based on the observation that flits of a flow are interleaved on a physical link due to the delivery of packets that belong to other flows only.

By substituting (2) into (1) we get:

$$t^i_j = \frac{l}{C_j - l \cdot \sum_{f \mid j \in \pi^f \wedge f \neq i} \lambda^f \cdot m^f} \quad (3)$$

The total transfer time, which is dominated by the hop with the longest delay, can then be written as:

$$T^i \simeq m^i \cdot \max(t^i_j \mid j \in \pi^i) \quad (4)$$

The above approximation does not capture the inter-link dependencies and is generally too optimistic, for medium and high loads. Wormhole links loads affects each other mainly due to the back-pressure mechanism: a flit must wait for the arrival of a credit for its virtual channel from the following link. Therefore, flit delivery time over a link (t^i_j) is affected by the delivery time in subsequent links of the flow's path. To reflect the effect of flit delay on other links we replace (3) by the following equation which accounts for these inter-link dependencies. Our simulations (section 6) show that this simple expression successfully estimates the resulting link delay:

$$\tilde{t}^i_j = t^i_j + \sum_{k \mid k \in \pi^i_j} \frac{l \cdot \Lambda^i_k}{C_k} \cdot \frac{t^i_k}{dist^i(j,k)} \quad (5)$$

where $dist^i(k,j)$ is the distance (measured in number of hops) between link j and k on the path of flow i. Formally written:

$$dist^i(k,j) = \left| \pi^i_j / \pi^i_k \right| \quad (6)$$

Equation (5) approximates the delay experienced by flits of flow i on link j by adding to the basic flit delay (t^i_j) a term that takes into account the accumulative effect of the delay of subsequent links along the path. This term is weighted by two factors: the links' distance from link j ($dist^i(k,j)$) and the factor by which they are being utilized by flows other than flow i itself ($l \cdot \Lambda^i_k / C_k$).

As explained above, the mean total transit time of each flow is calculated using the longest interleaving delay on its path. Therefore, (4) is replaced by:

$$T^i \simeq m^i \cdot \max(\tilde{t}^i_j \mid j \in \pi^i) \quad (7)$$

In section 6, we evaluate the quality of our approximated analysis against simulation studies.

5. Capacity allocation

Traditional wormhole networks employ links with uniform capacity set according to the traffic requirements. Since each link is not independently assigned the minimal capacity that is

required to meet the end-to-end delay requirement, some links are much faster than needed, consuming unnecessary power and area.

Given a network topology, a routing scheme, communication demands (in terms of packet generation rate and length) and QoS requirements, our algorithm adjusts the capacity of each link, striving to minimize the total allocated capacity while assuring that each flow meets its performance requirement.

In order to assign links capacities efficiently, we introduce the following notation:

T_{REQ}^{f} = *The required mean transit time for flow f*

δ = *The amount of capacity [bits/sec] added to the network on each iteration*

The capacity assignment minimization problem can now be formalized as follows:

Given:

F

$\forall f \in F: \quad m^f, \lambda^f$

$\forall f \in F: \quad T_{REQ}^{f}$

$\forall link\ j$, assign link capacities (C_j) s.t.:

$\forall f \in F: \quad T^f \le T_{REQ}^{f}$

$\forall link\ j: \quad \sum_{f \mid j \in \pi^f} \lambda^f \cdot m^f \cdot l < C_j$

$\sum C_j$ is minimal

Figure 2: Capacity assignment minimization problem.

Using this formulation, we suggest the algorithm described in figure 3 to allocate link capacities. The algorithm takes a greedy approach: The initialization phase allocates a minimal preliminary capacity to each link in the network. The main loop analyses each source-destination flow separately. It first uses the delay model (section 4) to approximate the flow's delay given the current capacity allocation vector (line 5). If the delay is larger than required (line 6), the algorithm allocates a small, predefined amount (δ) of extra capacity to the flow's path: It first assigns the extra capacity to each of the links along the path separately (lines 7-11) and approximates the resulting packet delay. It then searches for the link with the largest sensitivity to bandwidth addition, i.e. the link for which adding capacity results in the shortest delay overall (line 12). The extra capacity is added only to that link (line 13). When the algorithm terminates, all flows meet their required delay.

In practice, the analytical model does not capture all of the complex dependencies and effects in a virtual channelled wormhole network. As a result, the mean delay of a few flows may be under-estimated and the capacity of some links might be too small. Therefore, the resulting assignment is verified using network simulation, and some extra capacity may still be added to these flows' path.

```
      /*assign initial capacities*/
1)    foreach link e:
2)      C_e ← Σ_{f∈F: e∈π^f} λ^f · m^f · l
3)    end foreach

4)    foreach flow f ∈ F :
      /*evaluate current transit delay*/
5)      T^f ← Delay_Model (C, f)
6)      while (T^f > T_REQ^f )
          /*look for most sensitive link*/
7)        foreach e ∈ π^f :
8)          ∀j ≠ e: C̃_j = C_j
9)          C̃_e = C_e + δ
10)         T_e^f ← Delay_Model (C̃, f)
11)       end foreach
        /*get most sensitive link*/
12)     e' = argmin_e {T_e^f}
        /*increase its capacity*/
13)     C_e' = C_e' + δ
14)   end while
15)   end foreach
```

Figure 3: Capacity allocation algorithm.

6. Numerical results

In order to evaluate the delay model and to demonstrate the benefit of using the capacity allocation algorithm, we present the following two examples: The first example considers a homogeneous system, in which every module injects the same amount of bandwidth into the network and packets' destinations are chosen randomly with uniform distribution. While this is not typical to SoC, this scenario is very often used to evaluate wormhole networks. The second example represents a more realistic traffic for a SoC, exhibiting a non-uniform communication pattern: modules communicate with only a subset of all possible destinations and different flows have different bandwidth and different delay requirements. In particular, some modules send and receive packets from a single module (many-to-one and one-to-many patterns), emulating a SoC in which data is generated and destined at specific modules (for example , SoCs in which there is a main CPU or a single cache memory, or SoCs that use an off-chip DRAM memory). In practical SoCs, modules are also expected to be placed so as to minimize spatial traffic. Therefore, in the second example we place in proximity modules that exchange high bandwidth traffic.

For each example, we compare the analytical delay model with simulation results for a varying utilization factor, by using a wide range of uniform capacity allocation vectors. We then apply the suggested capacity allocation algorithm and present its benefit over uniform link capacities assignment. In both simulation and analysis, unit to router and router to unit links are assumed to have infinite capacity while inter router links have finite capacity.

We have implemented a tool that automatically assigns link capacities given the network topology, routing scheme, communication demands and QoS requirements. This tool, which uses the aforementioned delay model and capacity allocation algorithm (described in section 5), is to be used by the chip interconnect designer to minimize the network resources. The system was simulated using the OPNET modeler ([18]). The model includes all the complex dependencies between different flows in wormhole networks (due to particular virtual channel assignment schemes, finite router queues, etc.).

6.1. Homogeneous network example

The network comprises of a regular four by four, two dimensional mesh with XY routing. Links have identical capacities; packets of each flow are generated by a Poisson process, with a mean rate $\lambda = 1/4.8e-4$ [packets/sec]; packets consist of 500 flits, each flit is 16 bit long.

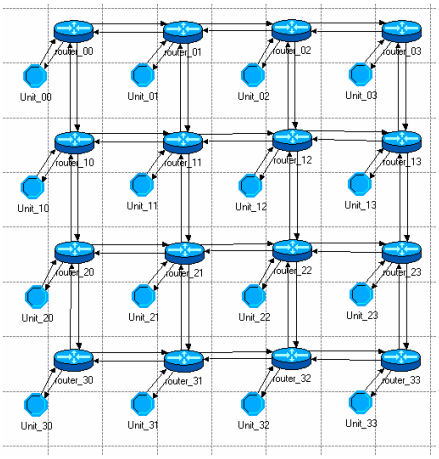


Figure 4: The homogeneous example: a four by four, two dimensional mesh, with 48 uni-directional inter-router links.

6.1.1. Delay analysis. As can be seen figure 5, though all flows inject the same bandwidth, the different distance and different aggregated load of links along their path results in a large diversity in the packet delays. Assuming that all flows have identical requirements, the mean transit delay of some flows is much lower than needed. This slack can be trimmed by a more efficient link capacity scheme.

Figure 6 compares the mean end-to-end packet delay of the analytical model with the simulation results as a function of the most utilized link utilization level (i.e. for a wide range of uniform capacity allocations). The analytical model closely follows the simulation results for a wide range of loads, way beyond the load that is expected to be found in a practical SoC (the mean absolute error reaches 8% when utilization is over 90%). The mean and mean absolute errors are presented in figure 7.

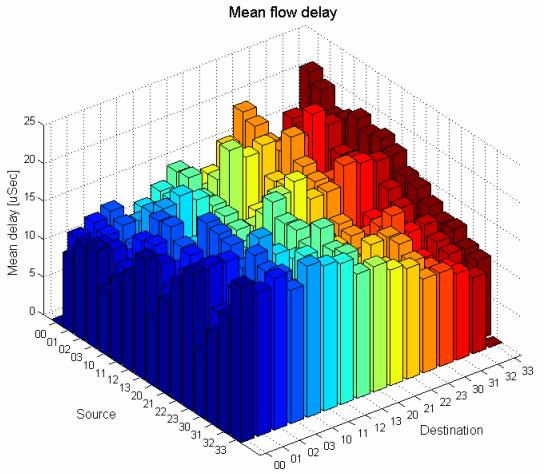


Figure 5: Mean delay in the homogeneous system. x and y axes mark the source and destinations modules respectably and z axis is the mean packet delay (longest delay is 2.5X longer than shortest one).

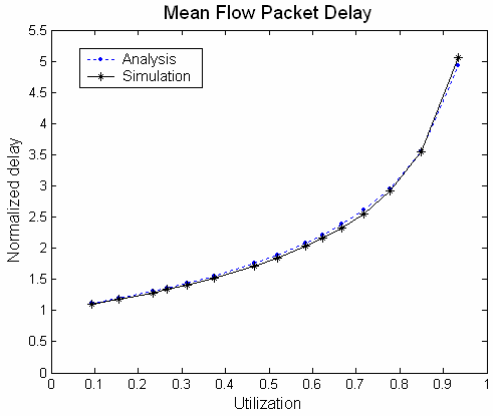


Figure 6: Model estimation and simulation results for the homogeneous system. x axis is the utilization of the most utilized link and y axis is the delay normalized by the delay in a zero loaded system (i.e. where no other flows exists).

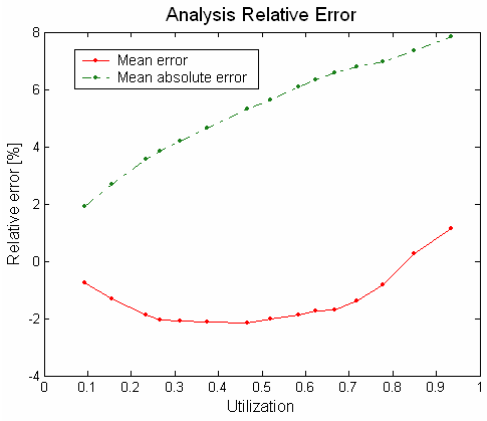


Figure 7: Mean and absolute error in the homogeneous system.

6.1.2. Capacity allocation results. In order to evaluate the suggested capacity allocation algorithm, we have compared it with uniform capacity allocation for a system with the same packet delay requirements (10uSec for each flow). By using simulations, we found that the total capacity needed in a uniform system is 74.4Gbits/sec (1.55Gb/sec per each one of the 48 inter-router links).

Figure 8 presents the capacities as assigned by the algorithm. A total budget of 69.0Gb/sec was used, which reflects a moderate 7.2% saving of resources. Figure 9 presents the resulting packet delay for each flow. Though less capacity is used, all flows still meet their delay requirement.

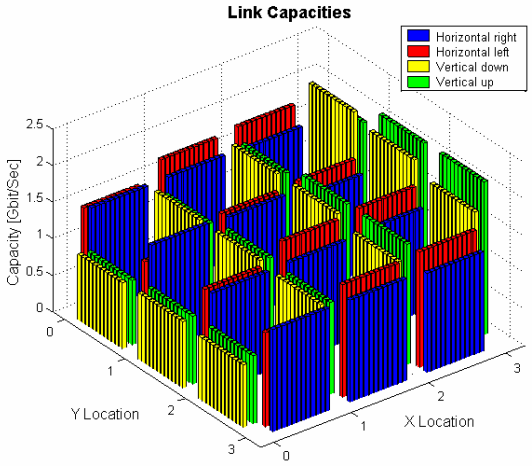


Figure 8: Links capacity as assigned by the capacity allocation algorithm for the homogeneous system.

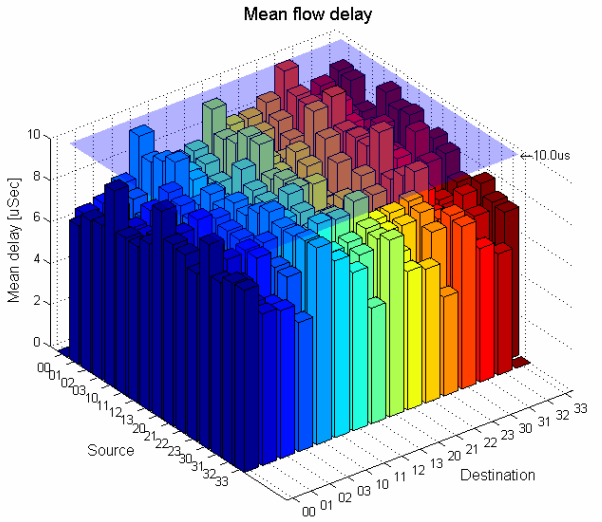


Figure 9: Flows delay in the homogeneous system when capacities are assigned by the capacity allocation algorithm.

As expected, the capacity allocation algorithm achieves a modest resources saving in the homogeneous example. Since all source-destination flows exist, it is impossible to reduce the capacity of a large number of links significantly without causing flows to miss their end-to-end delay requirement. Only the capacities of the leftmost links (both upward and downward) were considerably trimmed (e.g. links 00→10, 10→20 and 20→30 in Figure 8). This is because of their relatively light utilization following the XY routing scheme.

6.2. Heterogeneous network example

The heterogeneous network comprises of a regular three by four, two dimensional mesh with XY routing. Table 1 describes the different flows' characteristics and delay requirements.

Flow	Inter-arrival time [uSec]	Packet length [flits]	Total BW [Mb/sec]	Required delay [uSec]
00→01	16.67	500	480	5
01→12	66.67	500	120	10
01→10	66.67	500	120	10
01→00	66.67	500	120	5
01→02	16.67	500	480	10
01→21	5000	500	1.6	15
01→13	500	500	16	10
03→01	66.67	500	120	10
10→01	66.67	500	120	10
12→01	66.67	500	120	10
12→22	66.67	500	120	5
20→03	66.67	500	120	10
21→01	500	500	16	15
22→01	33.30	500	240	10
23→13	33.30	500	240	10

Table 1: Heterogeneous system

6.2.1. Delay analysis. Figure 10 compares the mean end-to-end packet delay of the analytical model with simulation results as a function of the most utilized link utilization factor. Since not all possible flows exist, and since flows have different bandwidth demands, this system is harder to analyze than the homogeneous system. However, the model properly approximates the resulting delays in the low to medium utilization range. As explained in section 4, NoCs are not expected to operate at high utilization, so the higher error rates in these regions do not present a problem. The mean and absolute errors are presented in Figure 11.

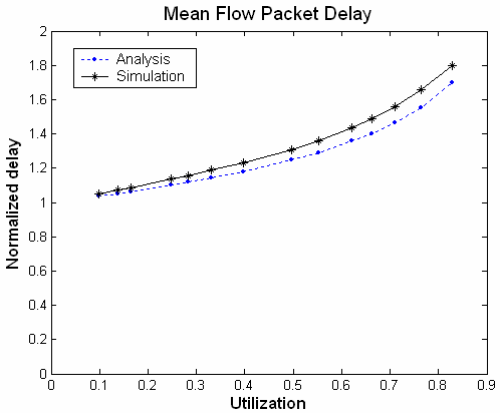


Figure 10: Analytical model estimation and simulation results for the heterogynous system. x axis is the utilization of the most loaded link and y axis is the delay normalized by the delay in a zero loaded system.

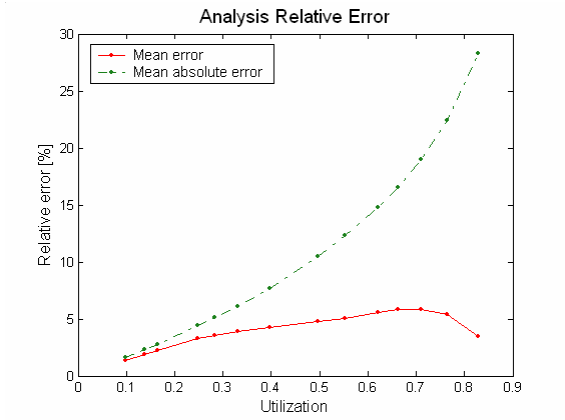


Figure 11: Mean and absolute error in the heterogeneous system.

6.2.2. Capacity allocation results. Similarly to subsection 6.1.2., we have compared the total budget assigned by the capacity allocation algorithm with a uniform assignment that meets the same requirements. Figure 12 presents the capacities assigned by the algorithm to each link. While the uniform assignment requires a total of 41.8Gb/sec, the algorithm used only 28.7Gb/sec, achieving a reduction of 30% in resources.

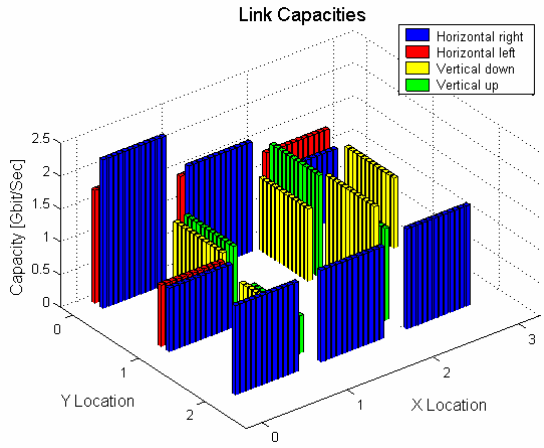


Figure 12: Links capacity assigned by the capacity allocation algorithm for the heterogeneous system.

Figure 13 presents the mean packet delay as approximated by the model and as evaluated by simulation. The figure also presents the maximal mean packet delay requirement for each flow. Though simple, the analytical model closely approximates the resulting packet delay and all flows meet their delay requirement.

Figure 14 compares the packets delay slacks (i.e. the difference between the actual packet delay and the delay requirement) in the two allocations discussed. Since in the uniform allocation all links are assigned identical capacity, the mean packet delays of some of the flows are much lower than needed. On the other hand, the capacity allocation algorithm adjusts links capacity independently, thus reducing the slack and improving efficiency. Note that flows may benefit from links with high capacity on their path, as it is needed to satisfy the requirements of other flows sharing the link. As a result slacks are still possible even in an optimal allocation.

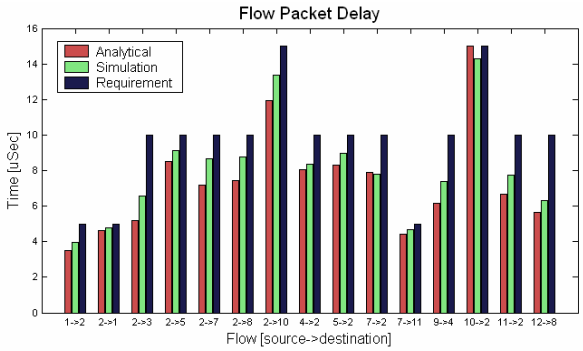


Figure 13: Capacity allocation results: mean packet delay as approximated by the model and as extracted from simulation vs. the maximal mean packet delay requirement.

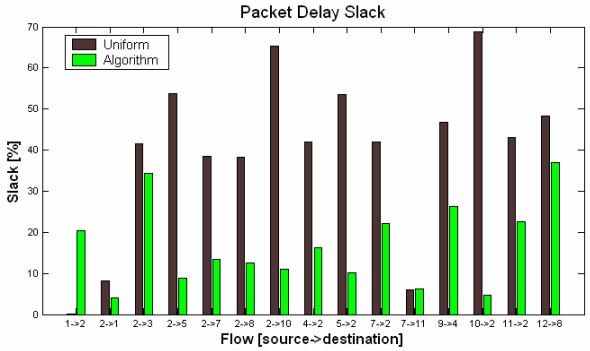


Figure 14: Mean packet delay slack as extracted from simulation, for uniform and algorithm-based allocation.

In this example, the capacity allocation algorithm was able to achieve a significant resource saving due to different bandwidth and requirements of the different flows, and due to the diversity in link loads. Since some flows have weaker requirements than others, it is possible to allocate relatively low capacity to some of the links, without causing any flow to violate its delay requirement. As a result, heterogeneous systems are more likely for a substantial resource saving than homogeneous ones.

7. Summary

Allocating links' capacities is an important phase in the design process of network on chip. A good assignment algorithm would allocate network resources efficiently so that QoS and performance requirements are met but total cost is minimized.

The paper includes two novel contributions: The first is a simple analytical model that captures virtual channelled wormhole networks with different link capacities and eliminates the need for repetitive simulations. The paper also suggests a capacity allocation algorithm that greedily assigns link capacities using the analytical model so that packets of each flow arrive within the required time. By using a design example, we showed the potential benefit of automated link capacity

allocation in a typical NoC-based SoC, where the traffic is heterogeneous and critical delay requirements vary largely.

8. References

[1] W. J. Dally and B. Towles. Route Packets, Not Wires: On-Chip Interconnection Networks. Proc. DAC, June 2001

[2] E. Bolotin, I. Cidon, R. Ginosar and A. Kolodny. Cost Considerations in Network on Chip. Journal of Systems Architecture, special issue on Network on Chip, Volume 50, February 2004, pp. 105-128

[3] S. Kumar, A. Jantsch, J. P. Soinien, M. Forsell, M. Millberg, J. Tiensyrjä, and A. Hemani. A Network on Chip Architecture and Design Methodology. Proceedings ISVLSI 2002

[4] W.J. Dally and C. Seitz. The Torus Routing Chip. Distributed Computing, vol. 1, no. 3, 1986, pp. 187-196

[5] E. Bolotin, I. Cidon, R. Ginosar, and A. Kolodny. QNoC: QoS Architecture and Design Process for Network on Chip. Special issue on Networks on Chip, The Journal of Systems Architecture, February 2004

[6] K. Goossens, J. Dielissen, O. P. Gangwal, S. G. Pestana, A. Radulescu, E. Rijpkema. A Design Flow for Application-Specific Networks on Chip with Guaranteed Performance to Accelerate SOC Design and Verification. DATE 2005.

[7] H. Sarbazi-Azad, A. Khonsari, and M. Ould-khaoua. Performance Analysis of Deterministic Routing in Wormhole k-ary n-cubes with Virtual Channels. Journal of Interconnection Networks, 2002, vol. 3

[8] S. Loucif, and M. Ould-khaoua. Modeling Latency in Deterministic Wormhole Routed Hypercube under Hot-Spot Traffic. The Journal of Supercomputing, March 2004

[9] C. Roche, P. Palnati and M. Gerla. Performance of Congestion Control Mechanisms in Wormhole Routing Networks. IEEE Infocom'97, Kobe, Japan, Apr. 1997

[10] J. Kim, and C. R. Das.Hypercube Communication Delay with Wormhole Routing. IEEE Transactions on Computers, July 1994

[11] R. I. Greenberg and L. Guan. Modeling and Comparison of Wormhole Routed Mesh and Torus Networks. IASTED, October 1997

[12] B. Ciciani, M. Colajanni, and C. Paolucci. Performance Evaluation of Deterministic Wormhole Routing in k-ary n-cubes. Journal of Parallel Computing, Dec. 1998

[13] J. T. Draper, and J. Ghosh. A Comprehensive Analytical Model for Wormhole Routing in Multicomputer Systems. Journal of Parallel and Distributed Computing, 1994

[14] W. Dally. Performance Analysis of k-ary n-cube Interconnection Networks. IEEE Trans. on Computers, June 1990

[15] J. Hu, R. Marculescu. Application-Specific Buffer Space Allocation for Networks-on-Chip Router Design. Proc. ICCAD 2004

[16] W. Dally. Virtual Channels Flow Control. Proc. ISCA, May 1990

[17] L. Kleinrock, Queuing Systems, volume 1: Theory, John Wiley & Sons Inc, New York, 1975

[18] OPENT modeler. (www.opnet.com)